Wavelet Molecular Orbitals for Quantum Chemistry

Building the electron-integral engine — NTU, Prof. Hsi-Sheng Goan's group

Po-Jen Wang

July 2026

Advancing Quantum Algorithms for Drug Design and Repurposing — S.-K. Chou, **P.-J. Wang**, J.-Y. Huang, Y.-H. Tung, S.-Y. Hsiao, H.-S. Goan.

Dept. of Physics & Center for Theoretical Physics, National Taiwan University.

What I owned, and why it is on this deck

My piece

The **electron-integral engine**: turn BIGDFT's wavelet orbitals into the h_{pq} and h_{pqrs} a qubit Hamiltonian needs — and work out what that costs *before* spending it.

- Ported the engine to the modern BIGDFT API
- Fixed three correctness bugs in it
- Put the hot loop (the Poisson solver) on the GPU
- Drove the N_2 and 70-atom ligand runs
- Built the downstream Qiskit / SQD integration

Why it belongs next to the QRE pitch

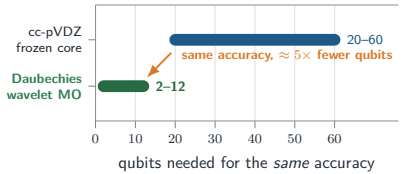
Count the operations. Take every symmetry the physics gives you free. Screen on locality. Put the hot loop on the right silicon.

Then decide whether the run is worth starting.

That is resource estimation — I just did it by hand, on one molecule.

Honest framing up front: the project **did not conclude** before I left, and no binding affinity was ever produced. What I can show you is the engineering and what it taught me about cost.

The basis set *is* the qubit budget

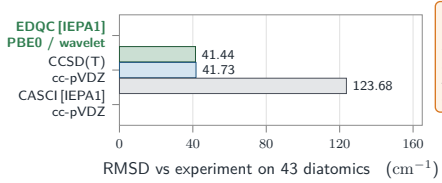


Why not just use STO-3G?

It is what almost every NISQ chemistry benchmark runs on, because it minimises qubits. The project's own verdict:

"the minimal basis set is chosen mere to meet the resource requirements… such a small basis often does not yield meaningful results."

For ranking drug candidates by binding energy, a cheap wrong number is worth nothing.



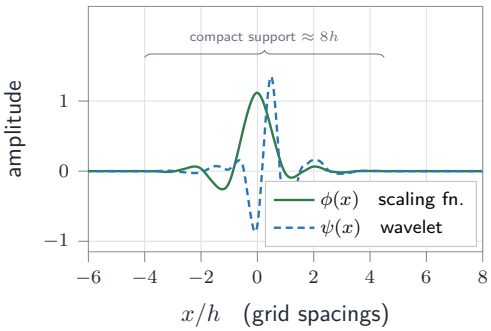
The lever. Change the basis, not the algorithm.

A real-space wavelet grid resolves the orbital where the molecule actually is, so a *minimal* set of wavelet MOs carries cc-pVDZ-grade physics — and the qubit count follows the number of orbitals, not the quality of each one.

Published results of the group this work builds on: Hong *et al.*, PRX Quantum **3**, 020360 (2022); Chou *et al.*, Phys. Rev. Research **5**, 043216 (2023).

Everyone benchmarks on STO-3G because it minimises qubits — and it is too inaccurate to rank drug candidates. Daubechies wavelet MOs get cc-pVDZ-grade accuracy at roughly a fifth of the qubits. **Change the basis, not the algorithm.**

What a Daubechies-16 basis actually is



least-asymmetric Daubechies, order $2m = 16$
exact for polynomials up to 8th order

3D basis at one grid point

$\phi^0 = \phi\phi\phi$ carried by **every** grid point

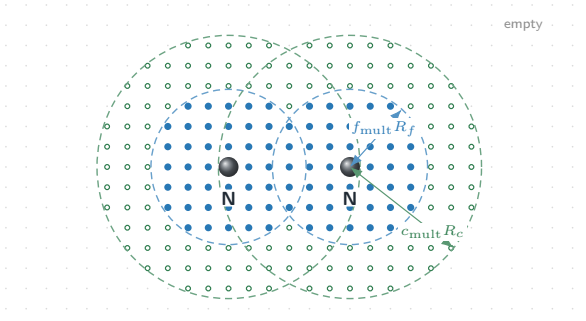
$\psi^1 = \psi\phi\phi$ $\psi^2 = \phi\psi\phi$ $\psi^3 = \psi\psi\phi$

$\psi^4 = \phi\phi\psi$ $\psi^5 = \psi\phi\psi$ $\psi^6 = \phi\psi\psi$

$\psi^7 = \psi\psi\psi$ added **only** in the fine region
$\Rightarrow 8\times$ the degrees of freedom

A basis with *compact support*: each function is exactly zero outside a few grid spacings. Orbitals get resolved where the molecule is, rather than as a sum of atom-centred Gaussians that stretch everywhere.

The basis follows the molecule



One grid, three regions

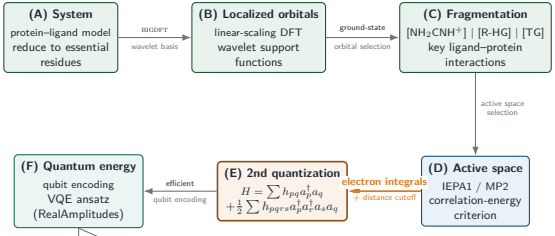
- **fine** — ϕ^0 *and* $\psi^{1..7}$
  8 DOF per point ⇒ half the effective spacing
- **coarse** — ϕ^0 only
  1 DOF per point
- **empty** — no basis functions
  vacuum costs nothing

The basis follows the molecule, not a box.
N_2 run: `hgrid 0.25`, `rmult [5.0, 8.0]`
⇒ 130,072 coefficients per orbital

R_c, R_f are per-species radii from the pseudopotential, with $R_f \ll R_c$ — so the fine sphere is the smaller one even though $f_{\mathrm{mult}} = 8.0 > c_{\mathrm{mult}} = 5.0$.

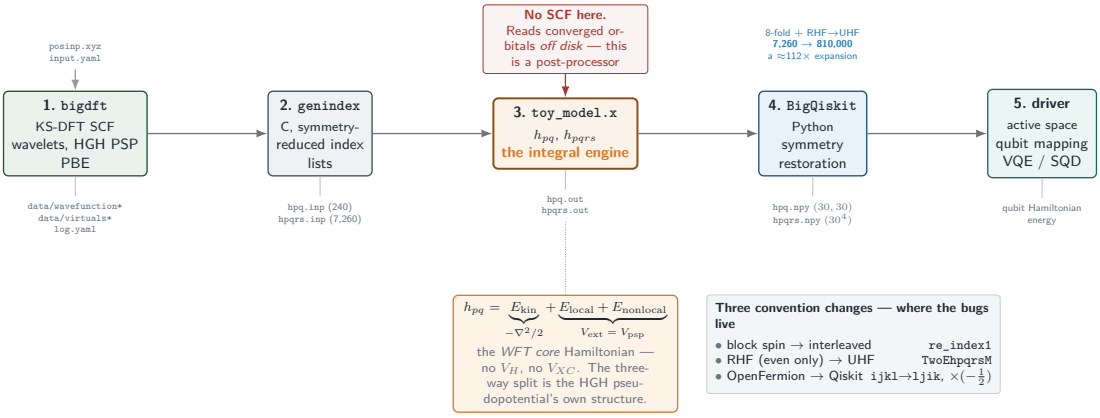
The fine region is *not* a denser mesh — it is the same lattice where each point carries 8 functions instead of 1. Vacuum costs nothing, and the accuracy knob is one number (`hgrid`), not a basis-set shopping trip.

The plan: six tasks, ending in a binding affinity



I owned the D→E link — the electron integrals, and the cutoff that makes them affordable.

What the code actually does



Worth saying plainly: `toy_model.f90` runs **no SCF** — it post-processes converged orbitals off disk. Two executables and a Python glue layer, and the joins between them are where the bugs lived. I fixed three of them.

The nugget: two-electron integrals out of a Poisson solver

What the qubit Hamiltonian needs

$$h_{pqrs} = \int\!\!\int \frac{\Psi_p(\mathbf{r}_1)\Psi_q(\mathbf{r}_1)\,\Psi_r(\mathbf{r}_2)\Psi_s(\mathbf{r}_2)}{|\mathbf{r}_1 - \mathbf{r}_2|}\, d\mathbf{r}_1 d\mathbf{r}_2$$

a 6D integral — and BIGDFT has no routine for it

What bigdft already solves every SCF step

$$V_H(\mathbf{r}) = \int \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r}' \quad \Longleftrightarrow \quad \nabla^2 V_H = -4\pi\rho$$

same kernel. The Poisson solver is already there, tuned, and parallel

The move: feed the solver a *transition* density $\rho_{rs} = \Psi_r \Psi_s$ instead of the physical ρ.

$$h_{pqrs} = \int \rho_{pq}(\mathbf{r})\, V_{rs}(\mathbf{r})\, d\mathbf{r} \qquad \text{with} \qquad V_{rs}(\mathbf{r}) = \int \frac{\rho_{rs}(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r}'$$

6D integral $\longrightarrow$ one Poisson solve + one 3D dot product

```
rhopq(:)  = phirr(:,ip)*phirr(:,iq)
rhors(:)  = phirr(:,ir)*phirr(:,is)
call Electrostatic_Solver(pkernel, rhors)  ! rho_rs → V_rs, in place
hpqrs     = sum(rhopq(:)*rhors(:))
```

BIGDFT subroutine `toy_model.f90`

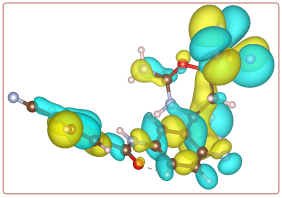
The cost, and the wall.
One `Electrostatic_Solver` call *per integral*, each on a $219 \times 195 \times 209$ box.

That single line is why the two-electron stage never finished for a 70-atom ligand — and why GPU-offloading *it* was the highest-leverage change available.

BIGDFT has no h_{pqrs} routine — Kohn–Sham never needs one. But its Hartree solver inverts exactly the right kernel: feed it a *transition* density and the 6D integral collapses to a dot product.

Why locality is the whole game

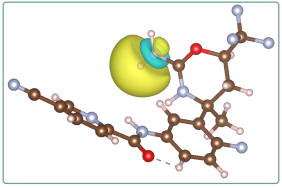
Canonical Kohn–Sham orbital #57



Cubic-scaling DFT. The orbital is an eigenfunction of the whole system, so it **smears over all 70 atoms**.

Every orbital overlaps every other orbital ⇒ es-

Localized support function #57



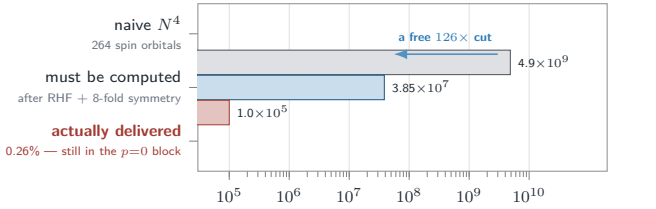
Linear-scaling DFT. The same electronic structure, re-expressed in orbitals confined to the **[NH_2CNH^+] fragment** that hydrogen-bonds to the aspartate.

Locality is not just cheaper — it makes the active-space selection sharper. Screen on the overlap $O_{pq} = \int |\Psi_p^* \Psi_q| \, d\mathbf{x}$: below ϵ, the integral is provably negligible and never computed. And because a localized orbital's two-electron integrals are *large* where it interacts and *small* where it does not, the IEPA1 correlation percentages separate more cleanly — the important orbitals become **more** important. Sparser Hamiltonian, fewer Pauli terms, better active space, from one change of representation.

Ligand 109 + ASP, 70 atoms, PBE / Daubechies wavelet, `hgrid 0.4` — the project's own BIGDFT renders (*insilico* report, Figs. 4–5). Linear-scaling variant: joint work.

Same molecule, same orbital index, two representations. Delocalized: everything overlaps, nothing vanishes. Localized: ≈**94%** of the integrals are never evaluated — and the active space gets *sharper* too.

What stopped us — and what I would pull



two-electron integrals for the 70-atom ligand (log scale)

Why it stopped. Every one of those 38.5 million integrals is a *separate Poisson solve* on a $219\times195\times209$ grid. The project's own projection for 264 spin orbitals: **≈600 days on 72 cores.**

The one-body side finished — `hpq.npy` is $(264, 264)$. It was never the problem.

The levers, and where each one got to

8-fold symmetry + spin — **shipped**

$h_{pqrs} = h_{qprs} = h_{rspq}$, and zero across spins $\Rightarrow$ ≈12% of the naive load

CUDA Poisson solver — **shipped — mine**

one dictionary flag, `'setup'//'accel'` → `'CUDA'`; offloads the hot loop itself

Locality screening — **prototyped**

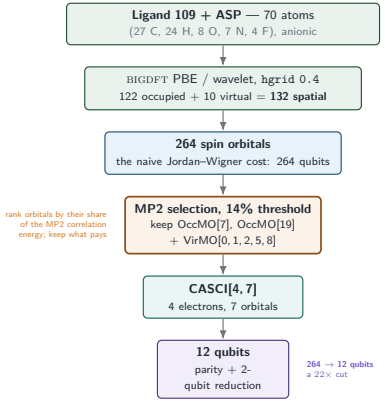
drop h_{pqrs} once the overlap $O_{pq} < \epsilon$ $\Rightarrow$ ≈94% fewer integrals to evaluate

Linear-scaling orbitals — **joint work**

localized support functions + distance cutoff $\Rightarrow$ $\mathcal{O}(N^3) \to \mathcal{O}(N)$

This is resource estimation. Count the operations, take every symmetry the physics gives you for free, screen on locality, put the hot loop on the right silicon — *then* decide whether the run is worth starting.

It did run end to end: 70 atoms → 12 qubits



What the run produced

qubits	12
Pauli terms	1,005
VQE parameters	24 (RealAmplitudes, 1 rep)
optimizer	SLSQP
VQE − exact	**11.6** $\mathrm{m}E_\mathrm{h}$

VQE against exact diagonalization of the *same* 12-qubit operator — so this number measures the optimizer, and it is meaningful.

Where the time actually went

reading hpqrs	**368.9 s**
running the VQE	70.0 s

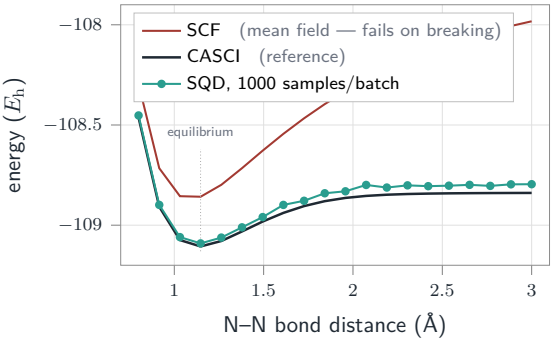
The classical integral handling outweighed the quantum algorithm by 5×. On this pipeline, the quantum part was never the bottleneck.

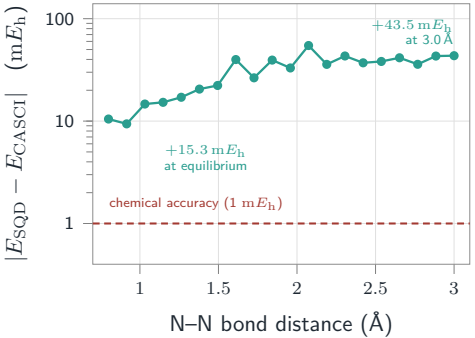
But the energies are not physical. The same log reports DFT-PBE = −462.41, HF = −2520.36, and a correlation energy of **−195** E_h — which is impossible; the real scale is ∼ −1 E_h.

The Hamiltonian was assembled from an *incomplete* h_{pqrs} set — 100,000 of 38.5 million entries, the rest silently zero. The plumbing ran end to end. The physics did not.

264 spin orbitals down to a 12-qubit Hamiltonian, and VQE matched exact diagonalization of that operator to 11.6 $\mathrm{m}E_\mathrm{h}$. **But that operator came from an incomplete integral set, so the energies are not physical** — which is exactly why the wall mattered.

So we went looking at SQD — and it did not rescue us either





The error is worst exactly where you wanted the quantum computer. SQD tracks CASCI near equilibrium and drifts as the bond breaks — i.e. it degrades as static correlation grows. **0 of 20** points reach chemical accuracy; the error runs $+9.4$ to $+54.7\,\mathrm{m}E_\mathrm{h}$.

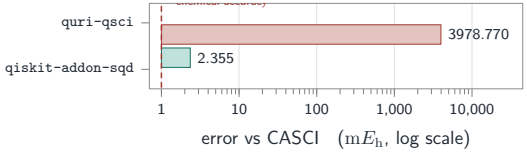
What this run is, exactly. PySCF / 6-31g integrals — *not* the BigDFT branch — and samples drawn from `generate_counts_uniform()`, a uniform-random stand-in for a QPU.

The notebook labels this curve `sqd-600`; the cell that produced it sets `SAMPLES_PER_BATCH = 1000`. Relabelled here.

What I did next, on my own

After the project: I went and benchmarked the hybrid stack myself

Read Reinholdt *et al.*, *Critical Limitations in Quantum-Selected CI* (arXiv:2501.07231) — then ran the comparison rather than taking it on faith. N_2 at $R = 1.0$ Å, HF ansatz, against a CASCI reference.



1,700× worse than the reference implementation. And the tell was in the convergence, not the number:

The bug: `quri-qsci`'s energy was *constant across every subspace size* — the selected-CI loop was not converging at all. A working QSCI must improve as the subspace grows; this one never moved.

Narrowed the discrepancy **19.5** E_h → **3.98** E_h by fixing API deprecations, non-Hermitian Hamiltonian construction, and a nuclear-repulsion sign convention.

Why I am telling you this one.

Two implementations of the same published algorithm, on the same molecule, disagreed by **4 Hartree**. Nothing in either paper would have told you which to trust.

You only find that out by running both against a reference and costing them honestly — which is exactly what the resource-estimation pipeline I want to build does, at scale and automatically.

Same instinct as the integral engine: count it, check it, then decide whether to spend the compute.

My own self-study, June 2025 — after leaving NTU, and not part of the project. Pinned stack: qiskit ≥ 2.1.0, qiskit-addon-sqd ≥ 0.11.0, quri-parts ≥ 0.22.1, pyscf ≥ 2.9.0.

What I took from it

What worked

- Wavelet MOs really do buy $\approx 5\times$ on qubits at equal accuracy
- Reusing the Poisson solver for h_{pqrs} — one routine, no new quadrature
- Locality: ≈94% of integrals never need evaluating
- The pipeline ran end to end on a 70-atom drug fragment

What did not, and why it matters

- The two-electron stage never finished — 0.26% of 38.5M integrals
- So the 109Asp energies are unphysical, and I can show you exactly how I know
- SQD plateaued 10–50$\times$ above chemical accuracy
- The project did not conclude before I left

The transferable part

Every one of those failures was a **resource** failure that a decent estimate would have caught up front — which is precisely what the multi-method QRE pipeline is for.

Backup: the group's published lineage

1. C.-L. Hong *et al.*, *Accurate and Efficient Quantum Computations of Molecular Properties Using Daubechies Wavelet Molecular Orbitals*, PRX Quantum 3, 020360 (2022) — wavelet MOs, H_2/LiH.
2. S.-K. Chou, J.-P. Chou, A. Hu, Y.-C. Cheng, H.-S. Goan, *Accurate harmonic vibrational frequencies for diatomic molecules via quantum computing*, Phys. Rev. Research 5, 043216 (2023) — 43 diatomics, IEPA1 active space, UCCSD vs RealAmplitudes.
3. Y. Shee *et al.*, *Qubit-efficient encoding scheme* Phys. Rev. Research 4, 023154 (2022) — the QEE encoding (implemented in this project by Y.-H. Tung and S.-Y. Hsiao, not by me).
4. J. J. M. Kirsopp *et al.*, *Quantum computational quantification of protein–ligand interactions*, IJQC 122, e26975 (2022) — the STO-3G baseline we were trying to beat.
5. L. Genovese *et al.*, J. Chem. Phys. 129, 014109 (2008) — bigdft and the Daubechies wavelet basis.

Attribution within the project: the integral engine (report Appendix A) was mine and Jing-Yan Huang's; the qubit-encoding work (Appendix B) was Tung's and Hsiao's; the linear-scaling variant was joint work.